

ENCN205

Applied Data Analysis for Civil and Natural Systems

LECTURE 3

Spatial Data Models



Slides and interactive demos (click or scan):
aardid.github.io/uc_205_gis_course

University of Canterbury | Te Whare Wānanga o Waitaha

Dr Alberto Ardid

Lecturer / Assistant Professor
Department of Civil and Environmental Engineering
University of Canterbury, New Zealand

E319 |
alberto.ardid@canterbury.ac.nz

"How do we represent the real world digitally?"

The real world is infinitely complex. Digital models are simplifications.

A river is continuous — but we store it as a line of coordinates

Elevation varies everywhere — but we store it as a grid of cells

A building has infinite detail — but we store it as a polygon outline

Today: the two fundamental models (vector & raster) and how they're structured in Python.

Vector Data Model — Geometry + Attributes

Recap from Lecture 1: vector = discrete features (points, lines, polygons)

Now we go deeper:

Each feature has two parts:

Geometry — the shape and location (coordinates)

Attributes — the descriptive data (name, material, length, date...)

Geometry types go beyond simple Point/Line/Polygon:

Multi-geometries: a single feature can have multiple parts (e.g. NZ = MultiPolygon)

Polygons can have holes (e.g. a lake with an island)

In Python: all geometry is handled by the Shapely library, wrapped by GeoPandas.

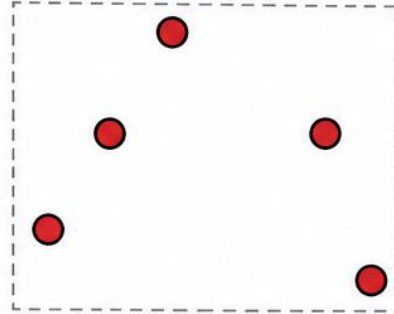
Geometry Types in the Vector Data Model

Point



Single (x, y) coordinate
e.g. sensor, borehole

MultiPoint



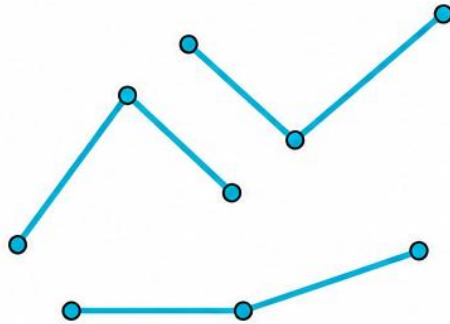
Multiple points = one feature
e.g. all boreholes at one site

LineString



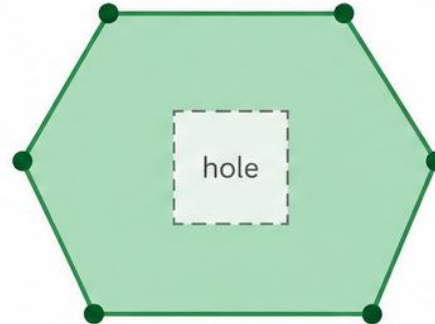
Ordered sequence of vertices
e.g. road, pipe, river

MultiLineString



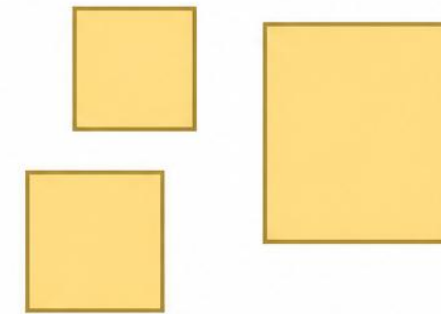
Multiple lines = one feature
e.g. discontinuous river

Polygon



Closed ring (can have holes)
e.g. building, lake

MultiPolygon



Multiple polygons = one feature
e.g. island group, split parcel

Every vector feature is one of these six types — your code must handle the right one.

The GeoDataFrame — Core Data Structure in GeoPandas

Attribute columns (pandas)					Geometry column (Shapely)
	id	name	material	length_m	geometry
0	1	Queen St	Concrete	245.3	
1	2	Park Ave	PVC	182.7	
2	3	Main Rd	Steel	310.1	
3	4	Elm St	Concrete	98.5	

Each row is one feature (one pipe)

Attribute columns

- Stored and handled by pandas
- Any data type (int, str, float, ...)
- Describe the properties of features

Geometry column

- Stores geometric objects (Shapely)
- Points, lines, polygons, etc.
- Defines the shape and location

Example geometry type



GeoDataFrame = pandas DataFrame + geometry column

Combines the power of pandas for data analysis with spatial geometry for location-aware analysis.

Key idea: a GeoDataFrame IS a pandas DataFrame with a special geometry column.

Everything you know from ENCN204 pandas still works — filtering, grouping, merging.

Always ask: 'does every row have a valid geometry?'

Scale Dependence -- Representation Changes with Scale

The same real-world feature can be represented differently depending on scale:

A river at 1:100,000 — a single LineString (centreline)

A river at 1:5,000 — a Polygon (showing width, banks, islands)

A building at 1:50,000 — a Point (centroid)

A building at 1:1,000 — a Polygon (footprint with exact shape)

A road at 1:250,000 — a single generalised LineString

A road at 1:2,000 — a Polygon showing lanes, medians, footpaths

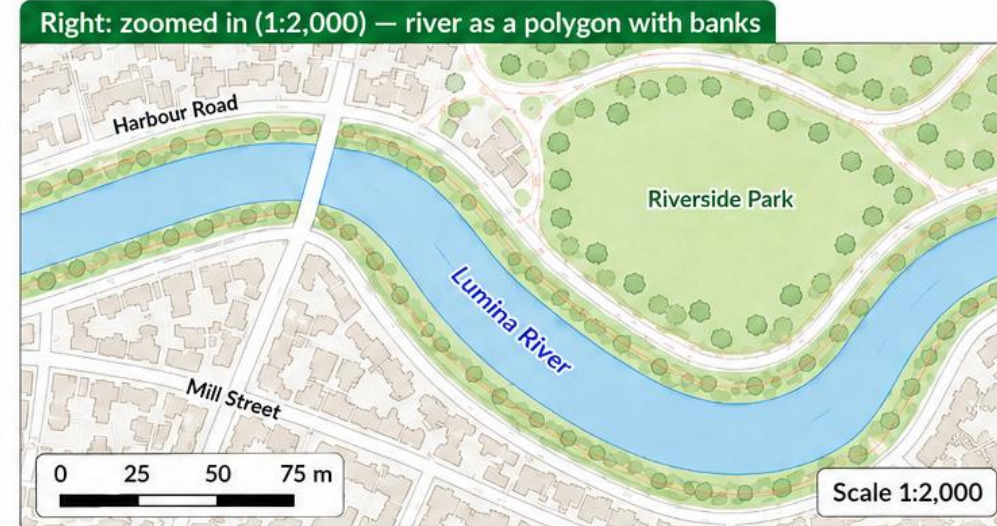
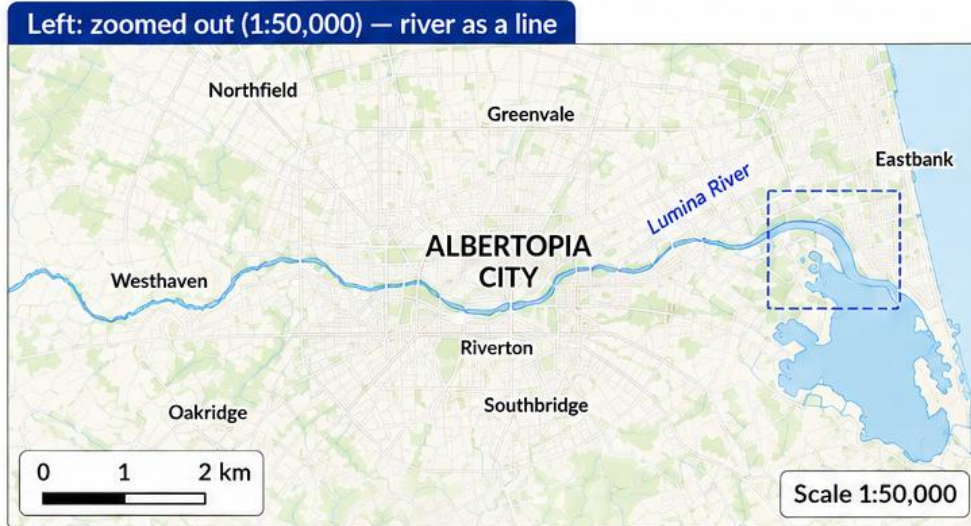
The geometry type you use must match the scale and purpose of your analysis.

Same Feature, Different Scales

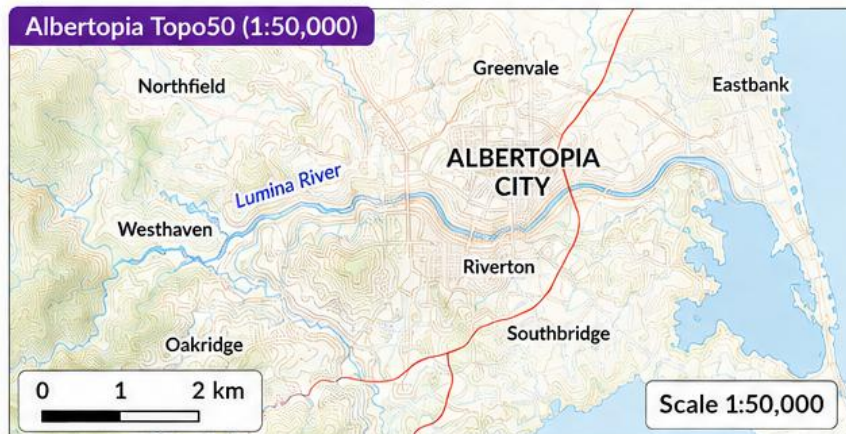
The same river can look very different depending on the scale and map generalisation.

💡 **Suggestion:** the Lumina River in Albertopia

- Left panel: zoomed out (1:50,000) — river as a **line**
- Right panel: zoomed in (1:2,000) — river as a **polygon with banks**



🔍 **Alternative:** Albertopia Topo50 for the same area



★ Key takeaway

Scale affects the level of detail, the representation of features, and the story the map tells.

Always choose the right scale and design for your purpose and your audience.

Raster Model in Depth

A raster is a grid of cells (pixels), each storing a value.

Key components:

Cell size (resolution): 1 m, 5 m, 25 m, 90 m...

Extent: the geographic area covered

Bands: one value per band per cell (DEM = 1 band, satellite = 3+ bands)

NoData value: marks cells with no valid data (often -9999)

CRS: the coordinate reference system

Affine Transform

Maps pixel (row, col) to geographic coordinates:

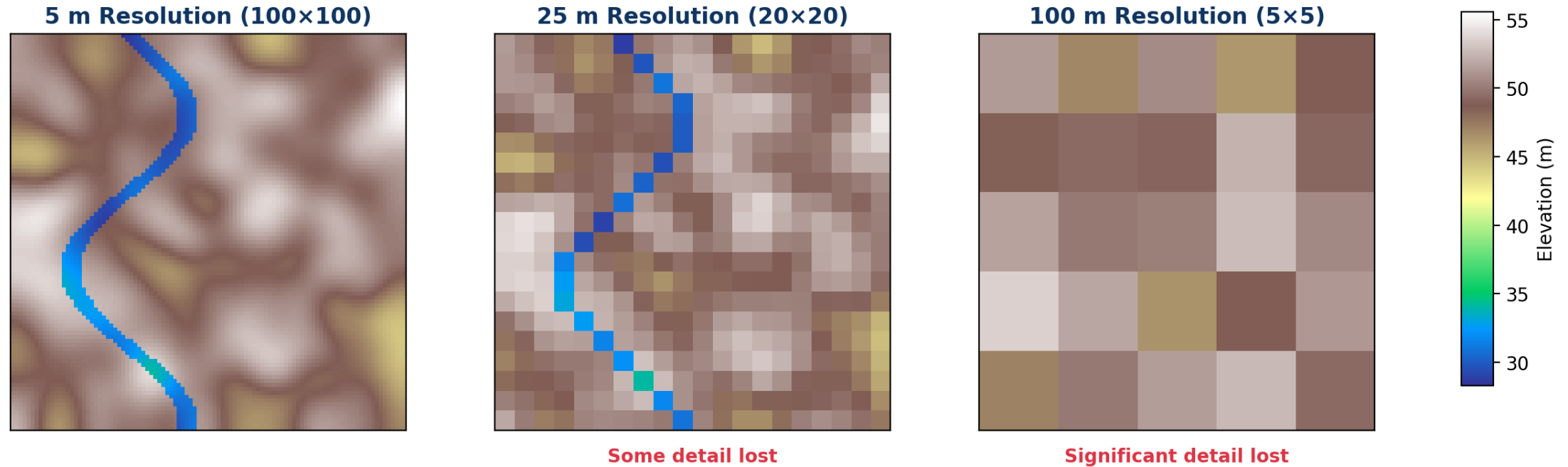
$$x = \text{origin_x} + \text{col} \times \text{pixel_width}$$
$$y = \text{origin_y} + \text{row} \times \text{pixel_height}$$

This is how the computer knows WHERE each cell is on Earth.

In Python: `rioxarray.open_rasterio()` reads rasters into `xarray DataArrays` with CRS and transform built in.

Resolution Matters — Same Area, Different Cell Sizes

Resolution Trade-Off — Same Area, Different Cell Sizes



Finer resolution = more detail but larger files and slower processing.

The resolution you need depends on the analysis — fitness-for-purpose applies to rasters too.

Always ask: 'what is the cell size?'

LIVE DEMO

[L03_VectorRaster.html](#)

Compare vector features (buildings, roads, rivers) with raster layers (DEM, satellite) on the same map. See how the same landscape is represented in both models.

Scan to open the demo on your phone



aardid.github.io/uc_205_gis_course/L03_VectorRaster.html
All slides & demos: aardid.github.io/uc_205_gis_course

Spatial Data Format Comparison

Capability score (0 = Poor, 10 = Excellent)

Capability	GeoPackage (.gpkg)	Shapefile (.shp/.dbf/...)	GeoJSON (.geojson)	GeoTIFF (.tif)	What it means
 Multi-layer	10 	3 	2 	3 	Can store multiple layers in one file.
 Large files	9 	7 	2 	10 	Handles large datasets efficiently.
 Web-friendly	5 	3 	10 	2 	Easy to use in web apps and APIs.
 Raster support	3 	0 	0 	10 	Native support for raster data.
 Attribute types	10 	4 	8 	2 	Supports rich and modern attribute data types.
 Open standard	10 	5 	10 	8 	Well-documented, open and widely adopted.



Key takeaway:

No single format is best for everything.

Choose the right format based on your data, size, and how you plan to use it.

-  **GeoPackage:** best all-rounder for vector data
-  **Shapefile:** widely used, simple, but limited
-  **GeoJSON:** best for web and APIs
-  **GeoTIFF:** best for raster data

For this course: GeoPackage for vector data, GeoTIFF for raster data.

Always ask: 'what format - and what did conversion lose?'

GeoPackage — The Recommended Format

Why GeoPackage?

Single file (.gpkg) — no more .shp + .dbf + .shx + .prj chaos

Multiple layers in one file

Full attribute types (date, boolean, long text)

No file size limits (Shapefile = 2 GB max)

Open standard (OGC) — not proprietary

SQLite database under the hood — fast queries

Shapefile limitations:

Multiple files required (.shp, .dbf, .shx, .prj, .cpg...)

Field names limited to 10 characters

No null values — uses workarounds

2 GB file size limit

Single geometry type per file

Shapefile is everywhere because it's old, not because it's good.

Reading Spatial Data in Python

Vector data — GeoPandas

```
import geopandas as gpd  
gdf = gpd.read_file("pipes.gpkg")  
gdf = gpd.read_file("buildings.shp")  
gdf = gpd.read_file("zones.geojson")
```

One function reads all vector formats — GeoPandas auto-detects the driver.

Raster data — rioxarray

```
import rioxarray  
dem = rioxarray.open_rasterio("elevation.tif")
```

Returns an xarray DataArray with CRS, transform, and NoData built in.

These two lines are the starting point of every lab.

Creating Geometry from Coordinates

What if your data is just a CSV with lat/lon columns?

```
import pandas as pd
import geopandas as gpd

df = pd.read_csv("sensors.csv")
gdf = gpd.GeoDataFrame(
    df,
    geometry=gpd.points_from_xy(df.longitude, df.latitude),
    crs="EPSG:4326"
)
```

`gpd.points_from_xy()` converts x/y columns to Point geometries.

You MUST set the CRS — otherwise GeoPandas doesn't know where the points are.

Always ask: 'which column is x, which is y?'

KEY CONCEPTS

Today's Takeaways

1. Vector: 6 geometry types (Point, MultiPoint, LineString, MultiLineString, Polygon, MultiPolygon)
2. GeoDataFrame = pandas DataFrame + geometry column — your core data structure
3. Raster: grid of cells with resolution, bands, NoData, and an affine transform
4. GeoPackage (vector) and GeoTIFF (raster) are the recommended formats
5. Most real projects combine both models — the choice depends on the analysis

NEXT LECTURE

Lecture 4: Coordinate Reference Systems

Engineering Decision: Choosing Vector vs Raster

Scenario: Flood assessment for a new housing development

You need:

Terrain — raster DEM (continuous surface, flow modelling)

Rainfall — raster (spatially varying precipitation)

Buildings — vector (exact footprints for exposure analysis)

Flood zone — vector polygon (regulatory boundary)

Pipes — vector lines (stormwater network)

Most real projects use BOTH models together.

The analysis pipeline: raster terrain → raster flow model → vector flood polygon → vector building intersection

Converting between models (rasterize / vectorize) is a key GIS skill.

Multi-Band Rasters — More Than One Value Per Cell

Single-band rasters:

DEM — one elevation value per cell

Land use classification — one class code per cell

Slope, aspect, curvature — derived from DEM

Multi-band rasters:

Satellite imagery (RGB) — 3 bands: Red, Green, Blue

Multispectral — 4+ bands (adds near-infrared, SWIR)

LiDAR returns — first return, last return, intensity

In Python:

Band 1: `data.sel(band=1)`

All bands: `data.values` has shape (bands, rows, cols)

In this course: mostly single-band rasters (DEMs).

Multi-band for awareness only.

Data Quality Revisited — Format and Model Matter

How format and model choices affect data quality:

Resolution: a 100 m DEM can't resolve a 10 m-wide channel

Generalisation: simplifying a polygon loses area accuracy

Conversion: rasterising a vector polygon introduces staircase edges

Format limits: Shapefile's 10-char field names can truncate metadata

NoData handling: if you ignore -9999 values in a DEM, your average elevation is wrong

Every modelling choice is a quality trade-off.

Document your choices: resolution, format, generalisation, conversion method.

A client gives you a Shapefile of pipe assets last updated in 2015. What do you do?

Consider:

What data quality checks from Lecture 2 apply here?

What are the Shapefile format limitations you should watch for?

Would you convert to GeoPackage? Why or why not?

How would you handle the 2015 timestamp — is it still usable?